between two strings, and keep track of the best result we have found, we can prune even more the number of nodes we traverse during search.

6.4.3 String Sorting

Burstsort[19] is a cache-efficient sort algorithm that works similarly to MSD (Most Significant Digit) radix sort. However, *burstsort* is cache-efficient and even faster than radix sort!

They both have the same asymptotic running time, `O(n * M)`, which is a theoretical lower bound for sorting `n` strings of length `M`, but *burstsort* results twice as fast, by exploiting *locality of reference* and better memory distribution.

Going into the details of this algorithm is out of the scope of this chapter, but to give you an idea of *burstsort* works, it dynamically constructs a trie while the strings are sorted, and uses it to partition them by assigning each string to a bucket (similarly to radix sort). The asymptotic cost, as mentioned, is the same as MSD's, because leading characters of each string are inspected once only. The pattern of memory accesses, however, makes better use of cache.

While MSD, prior to the bucket-sorting phase, accesses each string once for each character, *burstsort* accesses each string only once. The trie nodes, instead, are accessed randomly.

However, the set of trie nodes is much smaller than the set of strings, so cache is used more wisely.